Towards Privacy-Preserving and Practical Image-Centric Social Discovery

Xingliang Yuan, *Student Member, IEEE*, Xinyu Wang, Cong Wang, *Member, IEEE*, Anna Cinzia Squicciarini, *Member, IEEE*, and Kui Ren, *Fellow, IEEE*

Abstract—Images are becoming one of the key enablers of user connectivity in social media applications. Many of them are directly exploring image content to suggest new friends with similar interests. To handle the explosive volumes of images, one common trend is to leverage the public cloud as their robust service backend. Despite the convenience, exposing content-rich images to the cloud inevitably raises acute privacy concerns. In this paper, we propose a privacy-preserving architecture for image-centric social discovery services, designed to function over encrypted images. We first adopt the effective Bag-of-Words model to extract the "visual content" of users' images into respective image profile vectors. We then model the core problem as similarity retrieval of encrypted high-dimensional vectors. To achieve scalable services over millions of encrypted images, we design a secure and efficient index structure, which enables practical and accurate social discovery from the cloud, without revealing any image profile or image content. For completeness, we further enrich our service with secure updates, facilitating user's image update. Our implementation is deployed at an Android phone and Amazon Cloud, and extensive experiments are conducted on a large Flickr image dataset which demonstrates the desired quality of services.

Index Terms—Secure outsourcing, privacy-preserving, image-centric, social discovery

1 Introduction

With the proliferation of camera-equipped mobile devices, images are becoming one of the key enablers of users' connectivity at social media applications [2], [3]. Uploaded images contain rich information of their personal life, interests, favorites, and activities, which can effectively be utilized for social discovery. Flickr and Pinterest recommend social groups to users based on their uploaded images, while the users in these groups may fall into the similar topics or share the same interests [4]. Exploring the rich social media to automatically discover user relationships enables not just improved user browsing and social interaction experiences, but also better user targeting, enhanced personalized services, and even customized advertising [5].

Essentially, image-centric social discovery demands to evaluate and quantify content similarity over a very large number of images of users, e.g., over 95 million photos uploaded daily on Instagram in 2016 [6]. To effectively handle the large-scale computation and storage involved with the explosive growth of images, a trending treatment of many social media applications is to adopt the commercial public cloud platforms for their superior scalability and globally available services at relatively low cost. Among others, Instagram allows users' uploaded images to be stored directly in the Amazon S3 platform, rather than passing through their on-premise servers, and it further uses Amazon CDN to facilitate low latency services for users around the world [7].

Despite these benefits, exposing the content-rich images and the related social discovery results to the cloud service provider may harm the privacy of users. On the one hand, images usually contain user personal or sensitive semantic information by nature, e.g., locations, education background, family members, and interests, which can be harvested against users [8]. On the other hand, image-centric social discovery in an outsourced environment may bring significant privacy risks, because the public cloud is known to have a very broad attack surface [9]. Current protection mechanisms primarily focus on traditional access control, which only allows a user to decide the content scope of sharing. However, it can hardly resolve other privacy issues, such as uncontrolled disclosure or profiling, since images in the cloud are outside the control of users. The recent revelation of unsolicited large-scale surveillance program PRISM [10] further makes the threats of user privacy leakage more intimidating than ever.

In consideration of the above observations, we propose a new privacy-preserving image-centric social discovery service for public clouds. Unlike most existing privacy-preserving friend recommendation schemes that focus solely on image context, such as manually annotated image tags and captions [11], [12], our proposed design explores rich media and takes account of visual features of images for more effective and targeted friend suggestions [2], [3]. Our

- *X. Yuan, X. Wang, and C. Wang are with the Department of Computer Science, City University of Hong Kong, Hong Kong, China, and the City University of Hong Kong Shenzhen Research Institute, Shenzhen 518057, China. E-mail: xyuancs@gmail.com, xy.w@my.cityu.edu.hk, congwang@cityu.edu.hk.*
- *A.C. Squicciarini is with the College of Information Sciences and Technology, The Pennsylvania State University, State College, PA 16801. E-mail: asquicciarini@ist.psu.edu.*
- *K. Ren is with the Department of Computer Science and Engineering, The State University of New York, Buffalo, NY 14226. E-mail: kuiren@buffalo.edu.*

Manuscript received 17 Mar. 2016; revised 24 July 2016; accepted 12 Sept. 2016. Date of publication 15 Sept. 2016; date of current version 29 Aug. 2018. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below. Digital Object Identifier no. 10.1109/TDSC.2016.2609930

1545-5971 © 2016 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

rationale is that, consistent with the well-known "Homophily" theory [13], similar users are likely to connect with one another. In social media applications, we expect two users associated with images with similar content normally falling into similar interests and topics, and therefore are willing to connect. To realize image-centric social discovery, we first extract the "visual similarity content" of images into individual user profiles in the form of high dimensional vectors, following the visual Bag-of-Words model, i.e., a paradigm commonly used in content-based processing and retrieval [14]. Then we model the problem of comparing and quantifying image content similarity with privacy assurance as secure similarity retrieval over encrypted high-dimensional user image profiles.

Compatible with many existing social media applications, our proposed service also leverages the public cloud as the storage backend to host encrypted images. To preserve the usability, the cloud should still be able to perform fast and scalable similarity discovery over millions of encrypted images. According to the above requirements, we devise a secure and efficient index structure from building blocks of locality-sensitive hashing [15], cuckoo hashing [16], as well as recent advancements of searchable symmetric encryption [17], [18], [19]. The proposed index can facilitate practical and accurate social discovery with provable security. The resulting service provisions the cloud a controlled capability to local encrypted similar user image profiles, while protecting both the image content as well as the image profiles. Our design is expected to readily complement existing context-based techniques and serve for more advanced privacy-aware social media applications. The contributions are listed as below:

- Our system is the first to deliver the privacy-preserving image-centric social discovery service in an outsourced architecture of existing social media applications, which allows fast and scalable social discovery for millions of users with a large scale of encrypted images.
- Our proposed secure index structure guarantees the privacy protection on image content and user image profiles, while maintaining reasonably good social discovery accuracy, index space efficiency, low service latency, and constant service bandwidth cost.
- We extend the system functionality by supporting secure insertion and secure deletion for the updates of user image profiles. We also discuss the system compatibility with current image sharing services.
- We formalize the security definition and generalize the leakage definition in our system. We prove that our social discovery service and the user profile updates achieve semantic security against adaptive adversaries.
- We implement and deploy the proposed system on an Android phone and Amazon Cloud. Experiments over 1 million user image profiles created from the Flickr image dataset (MIRFlickr-1 M [20]) demonstrate the practical performance on system initialization, service latency, space efficiency, fine accuracy, and energy efficiency. The social discovery results are also shown to be consistent with human perception.

The paper organization: Section 2 states the system architecture, threat model, and preliminaries. Section 3 elaborates on our system design of privacy-preserving image-centric social discovery. Section 4 presents the formal security analysis of our system. Section 5 reports the performance evaluations, followed by Section 6 that overviews the related works. Finally, Section 7 concludes the whole paper.

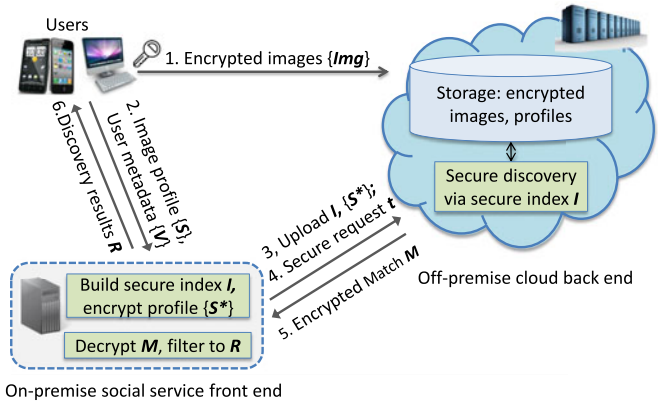


Fig. 1. Our proposed system architecture.

2 Problem Statement

2.1 System Architecture and Threat Model

The intuition for image-centric social discovery is to use image content to interpret the user interests. If two users post images on similar content or topics, such as the same places visited, similar types of food, pets, or plants they like, then they are likely to have similar interests, and therefore they may connect to possible friends, peers or even love partners [2], [3].

Our approach is to recommend friends by quantifying the similarity of visual content between users. The higher the similarity, the higher the chances that the two users would share similar interests. If two users' preferred images contain similar "visual words", then their corresponding image profiles will be closer to each other. For privacy preservation, the problem is thus how to find users with nearest image profiles with regard to any targeted user(s), among a large number of encrypted image profiles. Hereafter, we formulate it as secure similarity search over encrypted users' high-dimensional image profiles.

2.1.1 System Initialization and Service Flow

The high-level system architecture is illustrated in Fig. 1. It involves three cooperating entities: the users, the trusted social media application server as the on-premise service front end (SF), and the untrusted cloud server (CS) as the off-premise backend. Our service flow involves three phases: 1) *User data upload* shown in Steps 1 & 2: For privacy protection, each user client first encrypts all the images $\{Img\}$, then uploads them directly to CS. Meantime, each user client undertakes the preferred images to quantify user's image-centric interests via a small-size image profile S and derived metadata V, both of which will be sent to SF. 2) *Service front end initialization* shown in Step 3: Upon receiving all users' $\{S, V\}$, SF will fast group similar image profiles via an advanced secure index I to enable efficient and secure social discovery later at CS. 3) *Privacy-preserving social discovery* shown in Step 4, 5 & 6: For any targeted user

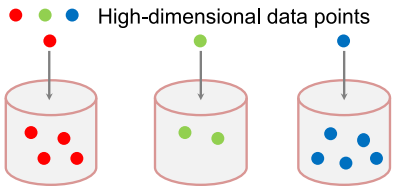


Fig. 2. Illustration of locality-sensitive hashing.

(s), *CS* is able to continuously process the secure social discovery request, in the form of secure trapdoor **t**, from *SF*, and returns encrypted matching result **M**, without seeing the images $\{Img\}$ or the users' image profiles $\{S\}$. The encrypted match **M** will be decrypted and filtered by *SF* to derive final social discovery result **R** for targeted users.

For practical considerations, our system supports subsequent user profile update securely and efficiently without affecting the correctness of social discovery service. In particular, the secure update operations are conducted over the secure index I between *SF* and *CS*. We remark that user clients in our system use their own keys to encrypt images they upload. To avoid performance bottleneck at *SF*, those encrypted images will straightly be uploaded to *CS* without passing through *SF*. Under such treatment, our system can still support existing image sharing functions, where users may employ different encryption techniques [21], [22] to enforce fine-grained access to their encrypted images. More details can be found in Section 3.5.

2.1.2 Threat Assumptions

We consider the security threats primarily from the "honest-but-curious" cloud, who dutifully follows the designated operations yet intends to infer users' encrypted image content, image profiles, and related information. We focus on protecting the confidentiality of those images and image profiles outsourced in the public cloud. We do not consider other attacks, such as those on data integrity, availability, etc., in this paper, which can be handled by various orthogonal mechanisms. In our system architecture, we assume that the service front end of social media applications and all users are trustworthy. We assume they can protect their secrets, such as keys and parameters, and all the communication channels are secure and authenticated.

2.2 Preliminaries

Our design involves locality-sensitive hashing and cuckoo hashing as building blocks. We provide a brief review of these notations next.

Locality-Sensitive Hashing. LSH is an algorithm to solve the approximate nearest neighbor search in high-dimensional spaces [15]. The idea is to hash the high-dimensional input data points via specially-designed LSH functions, where similar data points are hashed into the same bucket with a higher probability than distant ones as shown in Fig. 2. Let O be the domain of data points and D be the distance measure between points, then LSH is defined as follow:

Definition 1 (Locality-sensitive Hashing). *Given distance $r_1, r_2, r_1 < r_2$, and probability value $p_1, p_2, p_1 > p_2$, a function family $\mathcal{H} = \{h : O \rightarrow U\}$ is (r_1, r_2, p_1, p_2)-locality-sensitive if for any $o_i, o_j \in O$: if $D(o_i, o_j) \leq r_1$ then $P[h(o_i) = h(o_j)] \geq p_1$; if $D(o_i, o_j) > r_2$ then $P[h(o_i) = h(o_j)] \leq p_2$.*

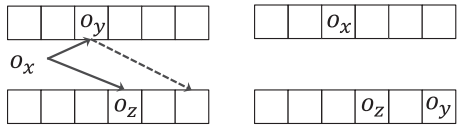


Fig. 3. Illustration of Cuckoo hashing: Data items are moved across two hash tables for better load balance.

For the accuracy of similarity search, multiple LSH hash tables are generally needed [23], [24] and each hash table is associated with a concatenation function containing numbers of LSH functions to enlarge the gap between p_1 and p_2 [15]. For a given query point q, one needs to hash q into buckets in hash tables to find all candidate points, and rank them based on their distances to q.

Cuckoo Hashing. Cuckoo hashing [16] is designed to build high-performance hash tables. It can disperse objects with hash collisions into different choices of hash tables for better load balance. The standard cuckoo hashing is defined below:

Definition 2 (Cuckoo Hashing). *Given two hash tables T_1 and T_2, both of them contain w units. Two independent and random hash functions $h_1, h_2 : O \rightarrow \{0, w-1\}$ are associated with T_1 and T_2, where O is the object domain. One object $o \in O$ can be inserted either in the bucket $T_1[h_1(o)]$ or the bucket $T_2[h_2(o)]$.*

If the hash collision happens, the object in the occupied bucket will be kicked away to another hash table iteratively until the kicked object finds space to stay. When more objects are inserted, the number of kick-away operations will increase. In practice, the standard cuckoo hashing can be extended with more than two hash tables.

3 Our Proposed Design

3.1 User Data Upload

The core of image-centric social discovery is to quantify visual similarity from users' images, and the users with higher visual similarity will have better chance to be recommended. On the one hand, our design borrows from the techniques of image processing to generate user image profiles, based on which, the visual similarity of users can be quantified. On the other hand, we consider adopting the "crowdsourcing" strategy to decentralize the tedious and potentially burdensome image preprocessing work to individual users, whose local devices like mobile phones generating the images have increasing computing power. Hence, each user preprocesses her own preferred images offline to facilitate the system initialization at *SF*. In particular, two tasks are assigned to each user's client:

3.1.1 User Image Profile Generation

We apply the Bag-of-Words model (BoW) [14] to generate the user image profile, which is shown effective in image similarity comparison and retrieval [2], [3]. The corresponding function is introduced as follows:

1) Client selects the preferred images $\{Img\}$ and extracts the image feature vectors, e.g., SIFT [25], SURF [26], etc.

2) For each selected image, the features are clustered into the visual words via K-means clustering according to the visual word vocabulary Δ, which is assumed to be pre-trained by *SF* and provisioned to each user client.
3) Each image is assigned with a BoW vector $W = \{w_1, \ldots, w_m\}$, where each element in W denotes the occurrences of the corresponding visual word and m is the cardinality of Δ.
4) Client then aggregates all the BoW vectors and normalizes it to create the user image profile vector $S = \{s_1, \ldots, s_m\}$, where s_i reflects the strength of user's preference on the ith visual word in Δ.
5) The images and the user image profiles are encrypted and uploaded directly to *CS*.

We note that the definition of image profile is not unique. As long as the profile effectively reflects the user's image-centric interests, it is applicable to our social discovery service. For examples, a direct adaption of term-frequency inverse-user-frequency (TF-IDF) can be used to create a visual word vector as the user profile [2]. Alternatively, the image features may directly represent the image-based interests of users [3].

Consequently, the distances between user image profiles become a metric to discover potential friends with similar image-centric interests. As known, in the distance measurement of high-dimensional vectors, two distance metrics are commonly adopted, i.e., euclidean distance and Cosine distance. As suggested in prior work in [27], both distance metrics are effective and the results are even similar for normalized vectors. They are consistent with human ideas of similarity. Without loss of generality, we choose Euclidean distance in our system design and implementation, where the visual similarity between user u_i and user u_j can be defined as $||S_i - S_j||$. We leave the effectiveness comparison against other metrics in future.

3.1.2 User Metadata Generation

For accurate similarity evaluation, "visual words" vocabulary Δ may contain more than hundreds or even thousands of words, making the user image profile S high-dimensional [14]. Hence, our treatment asks each user client to generate user metadata V, in the form of LSH hash values on S. The metadata will be uploaded to *SF* to facilitate the building of secure index for fast finding similar encrypted image profiles. The parameter $\mathbf{h} = \{h_1, \ldots, h_l\}$ denotes the set parameters of l concatenation LSH functions. After generating image profile S, each user client immediately computes her own metadata $V = \{h_1(S), \ldots, h_l(S)\}$ by applying LSH functions. Like Δ, the parameter $\mathbf{h}$ can be pre-shared.

Our system works transparently for existing social media applications such as Instagram that uses the public cloud as the backend storage [7]. Each user encrypts her images for privacy protection and also uploads them directly to *CS* without passing through *SF*. We will show these encrypted images are compatible with social data sharing services by exploring advanced encryption techniques in Section 3.5.

3.2 Service Front End Initialization

To facilitate *CS* to remotely perform privacy-preserving image-centric social discovery, the major task for *SF* is to build a secure similarity search index over encrypted high-dimensional image profiles. The question is how to make the design practical and be able to support millions of user profiles with high accuracy and strong security guarantees.

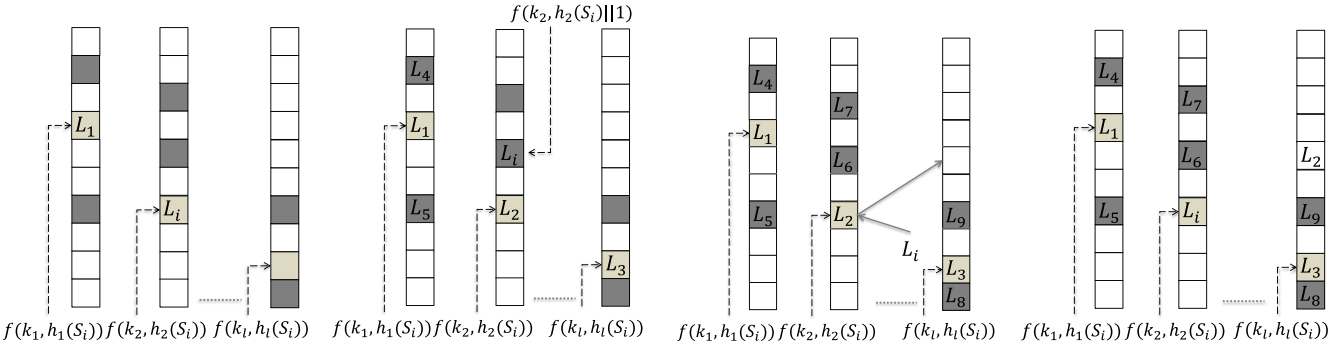
One straightforward approach is to first cluster similar user image profiles into the same bucket in LSH based hash tables. Then, a secure index can be built after encrypting all buckets and inserting random padding [28]. However, the imbalance of LSH [24] and the secure index requirement [17] make this approach suffer from unscalable index size $O(n^2)$ for n user image profiles, and large bandwidth cost $O(n)$ for each search request interaction. It is far from practical when meeting millions of users at social media applications.

Thus, we have to explore more advanced secure index designs that aim to take into account the practical performance parameters such as index size, index load factor (the ratio between the number of data items and the number of entries in the index), and security overhead, from the very beginning of the design. For this purpose, we resort to a recent index design [29] in the plaintext domain which utilizes the combination of LSH and cuckoo hashing for efficient similarity search over big data. As introduced, cuckoo hashing is known for building high-performance hash tables. It utilizes multiple hash tables and allows multiple hash locations for each data item. Moreover, data items can be "moved" among those positions during data insertions. Such a design provides a lot of flexibility on the imbalance of LSH and helps increase the index load factor [16]. The resulting index in [29] is shown to support good load balance, low search latency, and reasonable search accuracy. Yet, it supports only plaintext data without security.

3.2.1 The Proposed Secure and Efficient Index

How to leverage the aforementioned observation to design a provably secure index with all the salient features remains a challenging problem. Recall that in [29], the basic idea is to insert each data item into one of l initialized hash tables associated with l LSH functions. If any collision happens, we will use cuckoo-driven approach to kick away the existing data item and insert the very data item into another hash table from the remaining $l-1$ choices. To further resolve the hash collisions, the probing technique is also utilized [23], [24]. But for security guarantees, we need to do all the operations in a privacy-preserving manner. We achieve this by exploring a suite of carefully constructed security designs. Below we illustrate our design blueprint.

First, our design uses secure one-way functions, like PRF, to protect the sensitive LSH values while preserving the locality-sensitivity clustering. Note that LSH functions do not preserve the one-way property as pseudo-random functions. Thus, it is possible to reveal the input by observing LSH values. Second, a specially designed random probing technique is explored in index building to protect the similar image profiles with hash collisions. Specifically, the way searching for empty buckets to resolve collisions is in a random fashion. Finally, we encrypt data items stored in the hash table buckets via carefully designed random masks. Such masks can be extended for index update as shown later. Random padding is used to make all buckets indistinguishable. We illustrate our design in Fig. 4 and Algorithms 1, 2, and 3.



(a) Primary insert (b) Random probe ($d = 2$) (c) Cuckoo kick-way (d) Move L_2 from one to another

Fig. 4. The insertion phase in our proposed index construction.

Algorithm 1. Building Secure Index BuildIdx

INPUT: $K = \{k_1, \ldots, k_l, k_s\}$: secret key set; $\mathbf{S} = \{S_1, \ldots, S_n\}$: user image profile set; $\mathbf{V} = \{V_1, \ldots, V_n\}$: user profile metadata set.

OUTPUT: Secure index I.

```
procedure BuildIdx(K, S, V)
    N ← n/τ; w ← ⌈N/l⌉; I ← new {T}_l; Loop ← 0;
    for i ← 1 to n do
        Insert :
        if PrimaryInsert(L_i, V_i) = true then
            goto: Next;
        end if
        if RandomProbe(L_i, V_i, d) = true then
            goto: Next;
        end if
        while Loop ≤ MaxLoop do
            Randomly pick j ∈ [1, l];
            β ← f(k_j, V_i[j]); L_k ← T_j[β]; T_j[β] ← L_i;
            Loop++;
            goto: Insert; // insert L_k.
        end while
        rehash();
        Next :
    end for
    for i ← 1 to N do
        r_i = g(k_j, j||β); B_i = r_i ⊕ L_i; S*_i ← Enc(k_s, S_i);
    end for
end procedure
```

Let the image profile set be $\mathbf{S} = \{S_1, \ldots, S_n\}$ for n users. Let L_i be the identifier for S_i which enables CS to locate S_i. Let the user metadata set be $\mathbf{V} = \{V_1, \ldots, V_n\}$, where $V_i = \{h_1(S_i), \ldots, h_l(S_i)\}$ includes l LSH values on the input of S_i. Let $K = (k_1, \ldots, k_l, k_s)$ be secret keys via $\mathsf{GenKey}(1^\lambda)$ and $f : \{0,1\}^\lambda \times \{0,1\}^* \rightarrow \{0,1\}^u$, $g : \{0,1\}^\lambda \times \{0,1\}^* \rightarrow \{0,1\}^u$ be pseudo-random functions (PRF). $\mathsf{Enc}(k, \cdot)$ and $\mathsf{Dec}(k, \cdot)$ are symmetric key based encryption and decryption functions. Function $\mathsf{BuildIdx}(K, \mathbf{V}, \mathbf{S})$ builds the index I in the following phases.

- Setup phase:
  1) *Index initialization*: Set the index load factor $\tau \in (0, 1]$ as $\tau = n/N$, where N is the size of index I. Then create l hash tables. Each table T contains $w = \lceil N/l \rceil$ buckets, where each bucket is u-bit length. Set the random probe range d for each table.
- Insertion phase, for $1 \leq i \leq n$:
  2) *Primary insertion*: Insert L_i to one of l hash tables. Randomly pick $\alpha \in [1, l]$, and for j from α mod l to $(\alpha + l - 1)$ mod l, if bucket $T_j[f(k_j, h_j(S_i))]$ is empty, then insert L_i to $T_j[f(k_j, h_j(S_i))]$.
  3) *Random probe*: If the above l buckets are occupied, extra d random buckets for each table are probed. For δ from 1 to d, if bucket $T_j[f(k_j, h_j(S_i)||\delta)]$ is empty, then insert L_i to $T_j[f(k_j, h_j(S_i)||\delta)]$.
  4) *Cuckoo kick-away*: If no bucket is empty, randomly select j from 1 to l and kick away L_k located in $T_j[f(k_j, h_j(S_i))]$. Then place L_i to $T_j[f(k_j, h_j(S_i))]$ and re-insert L_k back via phases $1, 2, 3$ iteratively.
- Encryption phase:
  5) *Bucket encryption*: For $1 \leq i \leq n$, generate the random mask $r_i = g(k_j, j||\beta)$ for L_i located at $T_j[\beta]$. Then encrypt bucket $B_i = r_i \oplus L_i$.
  6) *Random padding*: Fill the remaining $N - n$ empty buckets with u-bit random strings with the equal length of B_i.
  7) *Image profile encryption*: For $1 \leq i \leq n$, call $\mathsf{Enc}(k_s, S_i)$ to output encrypted image profile S_i^*.

3.2.2 Discussion on Security and Efficiency

The security strength of our index lies in the fact that all buckets of hash tables are filled in a secure way, determined by pseudo-random functions, random padding, and random masking. Consequently, all encrypted bucket are indistinguishable. The secure index reveals nothing about the image profiles, the image content, and the user identifiers before initiating the social discovery service. Note that non-committing private key encryption, i.e., XOR L_i with a random mask r_i is applied to achieve the adaptive security. We will give the formal security proofs of our design in Section 4. The correctness of social discovery results is guaranteed because PRF transforms LSH values in a deterministic way. The users who share similar image-centric interests and have matched LSH values are still grouped together in the secure index.

The space complexity of our proposed secure index is $O(n)$, where n is the number of users in our system. No duplicate copies of identifiers are stored in the secure index. For a fixed number of users, the space consumption is only related to the index load factor. In Section 5.2, we show that our secure index is very compact, where its load factor can

achieve over 90 percent. We note that our design sacrifices a little accuracy for orders of space saving in contrast to the existing secure inverted index.

Algorithm 2. Subroutine PrimaryInsert

INPUT: L_i: image profile identifier; V_i: user metadata.
OUTPUT: boolean $true$ or $false$.

```
procedure PrimaryInsert(L_i, V_i)
    Randomly pick α ← [1, l];
    for j ← α mod l to (α + l − 1) mod l do
        β ← f(k_j, V_i[j]);
        if T_j[β] = NULL then
            T_j[β] ← L_i;
            return true;
        end if
    end for
    return false;
end procedure
```

Algorithm 3. Subroutine RandomProbe

INPUT: L_i: image profile identifier; V_i: user metadata; d: random probe range.
OUTPUT: boolean $true$ or $false$.

```
procedure RandomProbe(L_i, V_i, d)
    Randomly pick α ← [1, l];
    for j ← α mod l to (α + l − 1) mod l do
        for δ ← 1 to d do
            β ← f(k_j, V_i[j]||δ);
            if T_j[β] = NULL then
                T_j[β] ← L_i;
                return true;
            end if
        end for
    end for
    return false;
end procedure
```

The kick-away operation may cause multi-round insertion phases when most of buckets in the index are occupied. Specifically, we assign a large number for the maximum allowed insertion rounds defined as $MaxLoop$ at line 16 in Algorithm 1. In the worst case, the rehash operation at line 25 will be performed to generate new LSH function set and rebuild the entire index. In practice, a good choice of parameters (e.g., the load factor τ, the random probe range d, LSH parameters) can greatly reduce the number of kick-way operations which will be demonstrated in Section 5.2.

3.3 Privacy-Preserving Image-Centric Social Discovery

Our system delivers a privacy-preserving image-centric social discovery service via secure similarity search over encrypted image profiles. The methodology is built on the concept of secure index construction to match and retrieve a constant yet small amount of encrypted image profiles. *SF* first generates the secure social discovery query, which is a set of one-way transformed LSH values of a user's image profile. Upon receiving the secure query, *CS* locates the corresponding buckets and unmask them to reveal the social discovery candidates, i.e., the identifiers of similar user image profiles. After returning the matched encrypted image profiles, *SF* only needs to perform a light-weight distance ranking locally to obtain the final social discovery results. We state our service flow for a target user as follows:

1) *SF* transforms the user metadata V to a trapdoor $\mathbf{t} = \{(t_0^1, \ldots, t_d^1), \ldots, (t_0^l, \ldots, t_d^l)\}$, where $t_i^j = (\beta_i^j, r_i^j)$, $r_i^j = g(k_j, j||\beta_i^j)$, and β_i^j is specified by $f(k_j, V)$ for $i = 0$ and $f(k_j, V||i)$ for $1 \leq i \leq d$.
2) *CS* locates $l * (d+1)$ encrypted buckets $\{B_{i_c}\}$ specified by $\{\beta_{i_c}\}$, and unmask each $\{L_{i_c}\}$ via $r_{i_c} \oplus B_{i_c}$, for $1 \leq c \leq l * (d+1)$. With $\{L_{i_c}\}$, *CS* returns $l * (d+1)$ encrypted image profiles $\{S_{i_c}^*\}$, as the secure match M, to *SF*.
3) *SF* recovers $\{S_{i_c}\}$ via $\mathsf{Dec}(k_s, S_{i_c}^*)$ for distance ranking and filtering, and then pushes the recommended users $\mathbf{R}$ to the target user.

In the current design, *SF* has to be online to accept users' social discovery requests and generate secure trapdoors. To further relieve the burden at *SF*, one can adopt broadcast encryption to securely pre-share the keys to authorized users [17] so that users can directly send secure queries to the cloud. *SF* only needs to re-broadcast the new keys when some users are revoked.

3.3.1 Post Processing

Our design requires *SF* to retrieve all the matched results back for two reasons. First, those image profiles will be decrypted and ranked according to their Euclidean distances with regard to the query user's image profile. The results can be integrated with other friend recommendation algorithms for high-quality social discovery. For example, one may use Friend-of-Friend [30] approach to further filter the ranking results. The final candidates are recommended to the target user, who may want to make friends with acquaintances in addition to common image-centric interests [2]. *SF* can also filter the results with geographic locations of users' preferred images [3]. Persons captured in similar images may share similar interests, e.g., traveling to the same place. It helps provide a better social discovery service.

Second, the retrieved image profiles are indeed small-sized (less than a few hundred for properly selected l and d), and the distance ranking cost is marginal. We further note that this small bandwidth cost is constant when the user number grows, and will be sufficient for good social discovery accuracy. We will demonstrate these features in our experiment in Section 5.

3.4 Secure User Profile Update

In practice, users may occasionally upload new pictures to represent their updated social interests. Accordingly, our system needs to further support the corresponding update procedure on the secure index, while guaranteeing the correctness of social discovery service and the confidentiality of user image profiles and image content. When some user revises her preferred images, the image profile S and the metadata V will be updated. Before inserting the new profile, the outdated image profile should be deleted. Thus, we must require *SF* to retrieve the encrypted original image

profile for deletion, and then insert the new image profile back to the index. Accordingly, we propose two update operations, i.e., the secure deletion and the secure insertion.

The above operations also naturally support user registration and user cancellation services. When a new user registers at social media applications, the user client will generate the image profile and send it to *SF*. Then *SF* performs the secure insertion so that the new user can enjoy the privacy-preserving social discovery service. When a user wants to cancel the account, the client will notify *SF* to perform the secure deletion so that the user's information such as identifiers and encrypted image profiles will be removed from our system. To realize the secure update operations, we need to slightly revise the construction of the index bucket, which will be elaborated later.

3.4.1 Secure User Image Profile Deletion

When an encrypted user profile is removed from the system, the corresponding user identifier should not appear in the social discovery results later. Therefore, the identifier should be deleted from the secure index. Realizing the secure deletion is similar to requesting the social discovery service. Based on our index, the targeted identifier is stored within one of the buckets, determined by the one-way transformed LSH values of the corresponding user image profile.

In particular, we revise the way of bucket encryption as $B = (G(r) \oplus (L||V), \mathsf{Enc}(k_r, r))$, where r is defined as the state information and it is a random value, $G(\cdot)$ is PRF, k_r is another secret key kept by *SF*. For security, r should be newly picked each time whenever L needs to be re-masked at bucket B. To delete some L_i, *SF* generates the trapdoor to locate and retrieve the encrypted buckets. After that, *SF* unmasks them and empties the targeted bucket. To hide this bucket, all the accessed buckets are re-encrypted via fresh masks. The procedure of secure deletion is given below:

1) *SF* transforms the user metadata V to a trapdoor $\mathbf{t} = \{(t_0^1, \ldots, t_d^1), \ldots, (t_0^l, \ldots, t_d^l)\}$, where $t_i^j = \beta_i^j$, β_i^j is specified by $f(k_j, V)$ for $i = 0$ and $f(k_j, V||i)$ for $1 \leq i \leq d$.
2) *CS* locates $l * (d+1)$ encrypted buckets $\{B_{i_c}\}$ specified by $\{\beta_{i_c}\}$, and returns them back to *SF*.
3) *SF* recovers $\{r_{i_c}\}$ via $\mathsf{Dec}(k_r, \cdot)$ and unmasks $\{L_{i_c}\}$ via $\{G(r_{i_c})\}$ for $1 \leq c \leq l * (d+1)$. Then *SF* finds the bucket B_i storing L_i and replaces it with masking of a special label $\perp$: $B_i = (G(r_i') \oplus \perp, \mathsf{Enc}(k_r, r_i'))$, where r_i' is freshly picked.
4) *SF* re-encrypts each B_{i_c} of the remaining $l * (d+1) - 1$ buckets by using fresh r_{i_c}' and sends them to *CS*.
5) *CS* replaces the outdated buckets at the same positions, and removes the encrypted image profile S_i^* via L_i.

3.4.2 Secure User Image Profile Insertion

The secure insertion is to ensure that our system can find friends with similar image-centric interests adaptively when users update their image profiles. Each secure insertion groups the updated user and the users with similar image profiles. Secure insertion is less straightforward than secure deletion. Recall that, when building the index, inserting an identifier may involve multi-round kick-away operations. Therefore, the challenge is how to handle those kick-away operations securely and efficiently over the secure index.

If we keep on using the previous design for encrypted buckets, each kick-away operation involves the following five steps: (1) decrypt the l buckets located from *Primary insertion* and randomly select the kicked identifier; (2) retrieve and decrypt the image profile of kicked identifier; (3) compute the LSH values of image profile, i.e., the user metadata; (4) generate the trapdoor to retrieve the corresponding l buckets; (5) decrypt the above buckets to re-insert the kicked identifier or conduct the next kick-away operation. As a result, such design could be extremely expensive and involve several interactions between *CS* and *SF*.

We observe that the major overhead results from steps (2) and (3). The kick-away operation requires the related LSH values to find the available buckets and re-insert the kicked identifier, while they are not cached in our secure index. To eliminate such overhead, our design sacrifices the index space. In particular, the image profile's LSH values are concatenated to its identifier and encrypted by a random mask. $r \oplus (A||V)$ is stored, where $V = h_1(S)|| \cdots ||h_l(S)$. As a result, each kick-away operation can be implemented within one interaction between *CS* and *SF*. After decrypting the buckets in step (1), the identifier and LSH values are revealed. And *SF* can carry out step (4) immediately. After that, the new trapdoor is sent to *CS* to retrieve the buckets to re-insert the kicked identifier. The procedure of inserting an identifier L_i is given below:

1) *SF* transforms the user metadata V to a trapdoor $\mathbf{t} = \{(t_0^1, \ldots, t_d^1), \ldots, (t_0^l, \ldots, t_d^l)\}$, where $t_i^j = \beta_i^j$, β_i^j is specified by $f(k_j, V)$ for $i = 0$ and $f(k_j, V||i)$ for $1 \leq i \leq d$.
2) *CS* locates $l * (d+1)$ encrypted buckets $\{B_{i_c}\}$ specified by $\{\beta_{i_c}\}$, and returns them back to *SF*.
3) *SF* recovers $\{r_{i_c}\}$ via $\mathsf{Dec}(k_r, \cdot)$ and unmasks $\{L_{i_c}\}$ via $\{G(r_{i_c})\}$ for $1 \leq c \leq l * (d+1)$. Then *SF* finds the vacant bucket B_i storing $\perp$ and replaces it with $B_i = (G(r_i') \oplus L_i, \mathsf{Enc}(k_r, r_i'))$, where r_i' is freshly picked.
4) If no bucket is available, one of the l buckets in *Primary insertion*, say B_k, will be selected. *SF* inserts V_i into B_k via $B_k = (G(r_k') \oplus (L_i||V_i), \mathsf{Enc}(k_r, r_k'))$, and unmasks B_k to obtain V_k. To re-insert L_i, it re-runs step 1), 2), 3) & 4) iteratively until no kick-away operation happens.
5) For each interaction, *SF* re-encrypts each B_{i_c} of the remaining $l * (d+1) - 1$ buckets by using freshly picked r_{i_c}' and sends them to *CS*. Lastly, *CS* updates S_i^* via L_i.

3.4.3 Updated Secure Social Discovery Service

The corresponding privacy-preserving social discovery service flow needs to be updated since the bucket encryption is modified for secure insertion and deletion. For each service request, *SF* should first obtain the latest state information of buckets for the correctness of bucket decryption, which

introduces an interaction between *CS* and *SF* accordingly. Here, we present the updated service flow as follows:

1) *SF* transforms the user metadata V to a trapdoor $\mathbf{t} = \{(t_0^1, \ldots, t_d^1), \ldots, (t_0^l, \ldots, t_d^l)\}$, where $t_i^j = \beta_i^j$, β_i^j is specified by $f(k_j, V)$ for $i = 0$ and $f(k_j, V||i)$ for $1 \leq i \leq d$.
2) *CS* locates $l * (d+1)$ encrypted buckets $\{B_{i_c}\}$ specified by $\{\beta_{i_c}\}$, and returns $\{\mathsf{Enc}(k_r, r_{i_c})\}$ back to *SF*.
3) *SF* recovers $\{r_{i_c}\}$ via $\mathsf{Dec}(k_r, \cdot)$ and generates $\{G(r_{i_c})\}$ for $1 \leq c \leq l * (d+1)$.
4) *CS* unmasks each $\{L_{i_c}\}$ via $\{G(r_{i_c})\}$, and then returns the matched $l * (d+1)$ encrypted image profiles $\{S_{i_c}^*\}$, as the secure match $\mathbf{M}$, to *SF*.
5) *SF* recovers $\{S_{i_c}\}$ via $\mathsf{Dec}(k_s, S_{i_c}^*)$ for distance ranking and filtering, then pushes the recommended users $\mathbf{R}$ to the target user.

3.5 System Compatibility

Our system is compatible with the exiting image sharing service of social media applications. Encrypted data sharing in social networks has been proposed before. For example, Persona [21] adopts the attribute-based encryption. We give a usage example below to show how our design can utilize similar approaches for sharing encrypted images.

Each user first generates ABE private keys for her friends individually based on different attributes, e.g., family, co-worker, or classmate. Those attributes can be derived from user's existing access control policy. Then the client uses random keys to protect the uploaded images via symmetric encryption like AES. Meanwhile, the above image encryption keys are encrypted with ABE public key of the user. Only her friends with the proper ABE private keys can recover the image encryption key and decrypt the images. As shown in our system framework, each user can manage the friendship and mandate the access policies of encrypted images, which preserves the existing functions of social media applications.

Our design can also be utilized in social media applications using their own data centers. Having everything encrypted and stored at the server supports in-depth defense to mitigate threats from acute data breaches. Once the service is initialized, the decryption key is never sent to the server. Even if outside adversaries can occasionally steal the snapshot of the index, the user images, and the user profiles, they still do not see the underlying contents that are encrypted with a standard and strong level of security.

4 Security Analysis

In this section, we formally prove the security of our privacy-preserving social discovery system. Specifically, we first carefully categorize the possible leakage patterns for all the query operations between *SF* and *CS*, i.e., secure social discovery, secure insertion, and secure deletion. Then we follow the simulation based adaptive security definition, extended from recent searchable symmetric encryption (SSE) schemes [17], [19], to demonstrate that *CS* cannot infer any extra sensitive information from its view, i.e., a sequence of secure query operations and the corresponding results.

First, we quantify the information leakage in the view of *CS*, i.e., query pattern, access pattern, and intersection pattern. Then we show a probabilistic polynomial-time (P.P.T.) simulator can simulate an ideal index, output simulated query trapdoors, and response simulated results. To prove the security guarantee, we show that for a sequence of adaptive queries, the views from the real interactions between *SF* and *CS* and the simulated views cannot be distinguished. Leakage is formally defined as follows:

Definition 3 (Access Pattern AP). *Let the metadata set $\{V_1, \ldots, V_q\}$ be a sequence of q queries, $\{\mathbf{t}_{V_1}, \ldots, \mathbf{t}_{V_q}\}$ be the trapdoor set.* **AP** *is defined as q sets of buckets $(\{B_{V_1}\}, \ldots \{B_{V_q}\})$ and identifiers $(\{L_{V_1}\}, \ldots \{L_{V_q}\})$ for $\{\mathbf{t}_{V_1}, \ldots, \mathbf{t}_{V_q}\}$, where for each $\mathbf{t}_{V_i} = \{t^1, \ldots, t^l\}$, $t^j = \{t_0^j, \ldots, t_d^j\}$ is the sub trapdoor for hash table T_j and associated with $\{B_0^j, \ldots, B_d^j\}$.*

The access pattern **AP** includes the encrypted buckets in the index and the result identifiers, which are accessed via the secure trapdoor when conducting a secure operation. Such leakage to *CS* is not disclosed by the cryptographic primitives, but by the act of returning the appropriate results for our proposed secure operations. We note that for secure update operations, the mapping between the identifiers and the accessed buckets are not revealed because *SF* re-encrypts them and let *CS* passively conduct the update over the index.

Definition 4 (Similarity Query Pattern SP). *For a sequence of q queries,* **SP** *is defined as a symmetric matrix $\Pi_{q\times q}^{SP}$, where the element $\Pi^{SP}[i][j]$ is a binary vector ν that records the intersections $V_i \cap V_j$ for $1 \leq i, j \leq q$, if $V_i[m] = V_j[m]$, then $\nu[m] = 1$, otherwise $\nu[m] = 0$, for $1 \leq m \leq l$.*

In keyword search SSE schemes [17], [19], [31], [32], query pattern records the search queries for repeated keyword. While in our scheme, the trapdoors of queries will indicate the closeness of the query users' image profiles. Accordingly, we define the similarity query pattern **SP**. In particular, each trapdoor contains a number of subcomponents. And if two trapdoors have matched subcomponent (s), it means their corresponding image profiles have matched LSH values. Therefore, they are similar with high probability. Based on the definition of LSH in [15], the more LSH values are matched, the closer two image profiles are. Like existing SSE schemes, such leakage is also from the way of the service being used, which does not compromise the privacy of image profiles and image content.

Definition 5 (Intersection Pattern IP). *For a sequence of q queries,* **IP** *is defined as a symmetric matrix $\Pi_{q\times q}^{IP}$, where the element $\Pi^{IP}[i][j]$ contains l arrays recording the access intersection for q_i and q_j. Each array stores the intersections of identifiers $\{L\}$ and buckets $\{B\}$ for a given hash table T.*

Due to the intersections between trapdoors, the accessed buckets and the resulting identifiers have intersections as well. The intersection pattern **IP** is extracted from the access pattern **AP**. It is used for the simulation later in our security proof. For secure update operations, **IP** also only records the intersections of accessed buckets because the bucket decryption and re-encryption is performed at *SF*.

Definition 6 (View $\mathcal{V}$). *$\mathcal{V}$ is defined as $(\mathbf{AP}, \mathbf{SP}, \mathbf{IP}, I)$ for a sequence of q queries.*

View $\mathcal{V}$ is the precisely-defined information leakage observed by *CS* for a polynomial number of queries. The security of our privacy-preserving image-centric social discovery scheme is defined in the Theorem 1.

Theorem 1. *Our proposed scheme satisfies the adaptive semantic security, if there exists a P.P.T. simulator $\mathcal{S}$ and a P.P.T. adversary $\mathcal{A}$, for a sufficiently large γ and every positive polynomial function $p(\cdot)$, we have $P[\mathcal{A}(\mathcal{V}) = 1] - P[\mathcal{A}(\widetilde{\mathcal{V}}) = 1] < \frac{1}{p(\gamma)}$, i.e., $\mathcal{V}$ and $\widetilde{\mathcal{V}}$ are computationally indistinguishable.*

Proof. We will prove that the P.P.T. $\mathcal{S}$ can simulate an index $\widetilde{I}$ from the view $\mathcal{V}$, and then adaptively simulate the view $\widetilde{\mathcal{V}}$, which is indistinguishable with the real view $\mathcal{V}$. The steps of simulating $\widetilde{\mathcal{V}}$ are presented as follows:

- To build the simulated index $\widetilde{I}$, $\mathcal{S}$ generates N random buckets stored in l hash tables, where N is the size of I and l is the number of hash tables in I. Recall that the buckets in real index I contain the masked identifiers or the equal length random padding. Therefore, $\mathcal{S}$ generates the random string $\widetilde{B_i}$ such that the bit length of simulated buckets is equal to the length of real buckets, i.e., $|\widetilde{B_i}| = |B_i|$.
- To adaptively simulate the indistinguishable view $\widetilde{\mathcal{V}}$ for social discovery queries, $\mathcal{S}$ should simulate the trapdoors, the accessed buckets, the resulting identifiers and the corresponding encrypted image profiles. To adaptively simulate the trapdoors, $\mathcal{S}$ should create them consistently with **SP**, otherwise, the simulation will be detected by adversary. For the first query q_1, $\mathcal{S}$ simulates the trapdoor for l hash table one by one. The simulated trapdoor $\widetilde{\mathbf{t}}$ contains the same number of subcomponents as the real trapdoor $\mathbf{t}$, where each subcomponent is some random string with the equal length of real ones. After that, $\mathcal{S}$ uniformly samples $\beta' \in [0, w)$ to locate the accessed bucket $\widetilde{B_i}$. From the access pattern **AP**, $\mathcal{S}$ uses the identifiers $\{\widetilde{L}\}$, which are identical as the identifiers $\{L\}$. For an identifier $\widetilde{L_i}$, there must exist a random $\widetilde{r}$ where $\widetilde{L_i} = \widetilde{r} \oplus \widetilde{B_i}$, so $\mathcal{S}$ can correctly simulate the random mask via $\widetilde{r} = \widetilde{L_i} \oplus \widetilde{B_i}$. Afterwards, $\mathcal{S}$ generates the simulated ciphertext $\{\widetilde{S^*}\}$ with the same size of $\{S^*\}$, where each $S_i^{*'}$ is a random string and subject to $|\widetilde{S^*}_i| = |S_i^*|$. For the subsequent query q_{i+1} $(1 \le i \le q-1)$, $\mathcal{S}$ adaptively simulates $\widetilde{\mathbf{t}_{i+1}}$ from **SP** and **IP** from the view of the previous i queries. Explicitly, $\mathcal{S}$ still first simulates the trapdoors for l hash tables one by one. If the subcomponent $\widetilde{t^j}$ has not appeared before, $\mathcal{S}$ adopts the same approach to generate the random string $\widetilde{t^j}$. Otherwise, $\mathcal{S}$ copies the same $\widetilde{t^j}$. Likewise, from **IP**, $\mathcal{S}$ randomly assigns the buckets for ones that have not appeared before and uses the same buckets for the accessed ones. Besides, $\mathcal{S}$ generates random strings for newly retrieved ciphertexts and uses the same ciphertexts for the previously retrieved ones. For the result identifiers, they are identical to the ones in **AP**. Therefore, the simulated queries are consistent due to the matched intersections with the real queries.
- To adaptively simulate the indistinguishable view $\widetilde{\mathcal{V}}$ for secure update operations, $\mathcal{S}$ follows the similar approach for social discovery queries to simulate the interactions between *SF* and *CS*. For secure deletion, $\mathcal{S}$ generates random strings for the trapdoor $\widetilde{\mathbf{t}}$ via **SP**, and randomly assign buckets via **AP** and **IP**. The above simulation is exactly the same as the simulation for social discovery queries. Note that, for the accessed buckets, $\mathcal{S}$ will generate random strings to replace the existing ones to simulate the bucket re-encryption. For secure insertion, kick-way operations may involve multi-round interactions between *SF* and *CS*. Because each interaction accesses the same number of encrypted buckets and re-encrypts them too, it can be simulated in the same way as the secure deletion.

Due to the pseudo-randomness of secure PRF, $\widetilde{\mathbf{t}}$ and $\mathbf{t}$, $\widetilde{B}$ and B are indistinguishable. Due to the semantic security of symmetric encryption, $\widetilde{S^*}$ and S^* are computationally indistinguishable. Therefore, $\widetilde{\mathcal{V}}$ and $\mathcal{V}$ cannot be differentiated by a P.P.T. adversary. □

4.1 Discussion on Controlled Leakage

Similar to the existing SSE schemes [17], [19], [31], [33], the controlled information leakage is from the manner of service flows and required for correct query operations. *CS* learns **SP**, **AP**, and **IP** in order to return the correct results. Such leakage is from the fact that all trapdoors are deterministic [34]. Based on the leakage, several attacks on searchable encryption are proposed to compromise the query and the dataset [34], [35]. But those attacks require the adversary knows the full or partial knowledge of the queries and the dataset. And as indicated in [35], adding random padding on the index and the queries can effectively defend against such attacks with more introduced space and time costs. To mitigate such statistical information leakage, another trick which can be applied is to batch the social discovery requests from multiple randomly selected users to obfuscate the view of *CS* [31].

The secure update essentially conforms to the same procedure as secure social discovery. Hence, the secure deletion and the secure insertion do not compromise the privacy of user image profiles and image content. However, they do leak the closeness between the updated image profiles and some relevant profiles in specific conditions, i.e., the trapdoors of secure insertion and secure deletion have matched subcomponents with the trapdoors of social discovery queries which have been conducted before. After the update operation, *CS* may know similar users of the target one without requesting a social discovery service. Such

TABLE 1
Time Cost on LG Nexus 4

User profile time (s)	Meta data computation time (s)				
	l=10	l=20	l=30	l=40	l=50
26.5	0.015	0.017	0.019	0.023	0.026

TABLE 2
Energy Consumption on LG Nexus 4

Compute (mj)	Meta data & user profile upload (mj)					
	l=10	l=20	l=30	l=40	l=50	Profile
18.1	720	719	757	719	871	757

leakage also appears in existing SSE schemes with update operations [19], [31], [33]. Recently, an SSE scheme in [32] partially mitigate the update leakage by periodically rebuilding a hierarchal index to scramble the access pattern. Yet, it requires an extra client storage, relatively expensive communication and computation cost.

5 Experimental Evaluation

5.1 Experimental Setup

We implement the functions on *SF* and *CS* by *C++*, and the functions on the client by *Java*. We deploy the user client on an Android phone, i.e., LG Nexus 4 with quad-core 1.5 GHz ARM CPU and 2 GB RAM, and deploy *SF* and *CS* on two Amazon AWS (EC2) "c3.4xlarge" instances with 16-core 2.8 GHz vCPUs and 30 GiB memory respectively.

Our system uses the computer vision library OpenCV4Android [36] for visual word vocabulary training, image feature extraction, and image profile vector generation. Our experiment uses Speeded up Robust Features (SURF) to extract features with a Determinant of Hessian interest point detector [26]. The interest point is referred to as a local salient patch, and each is associated with a 64-dimensional feature vector. Then we use the MIT E2LSH library [37] to train and derive the parameters of LSH functions. We also use OpenSSL library to realize the symmetric encryption via AES-128 and the PRF via SHA-2. From the Flickr image collection with 1 million images, MIRFLICKR-1M [20], we choose 140,000 images with popular annotations as our experiment image dataset. We randomly pick 10 percent from our image dataset to build a 1,000-word vocabulary via K-means clustering [14]. Afterwards, 1 million users are created, where each is assigned with five distinct images independently and randomly selected from the remaining 90 percent of the image dataset.

5.2 System Evaluation

We now report our system efficiency and scalability from a number of performance factors, including client computation and energy costs, index space overhead, bandwidth overhead, service latency, and service accuracy. We will also compare our design with an existing work [28] denoted as KIK12, which builds the secure index directly based on LSH hash tables. Further, we give a visual evaluation of our social discovery results to show our security design does not affect the quality of service.

5.2.1 User Client Overhead

To demonstrate our design can be applied to mobile social media applications, we evaluate the computation and energy consumption on an Android phone. Our system requires each user client to extract the image features, generate the user image profile, and compute the metadata, i.e., LSH values. In this experiment, the user client processes a small number of preferred images, i.e., five images. As shown in Table 1, the user profile generation including feature extraction and aggregation dominates the time costs on the mobile phone, which takes around 26 s. With the increased number of LSH functions l, the meta data generation time increases slightly, but computing 50 LSH functions only needs 26 ms, which is still negligible to the user profile generation. Besides, the storage cost is 1.03 MB for visual word vocabulary, which is distributed initially to user client. We argue that the increasingly powerful mobile phones and tablets can handle the above overhead efficiently.

We also evaluate another critical factor, i.e., the energy consumption at the mobile client. Table 2 reports the energy consumption on computation and communication. Specifically, we use an energy profiling tool called "Power Tutor 2" [38] to estimate the energy costs. The communication costs are introduced by the uploaded user profile and the meta data. We use 4bytes to represent each LSH value, and thus it takes 0.2 KB for $l = 50$. And each 1,000-dimensional vector costs 4 KB. As the size of uploaded data is small, the energy consumption on communication is primarily caused by establishing the connection to the cloud. For the energy cost of computation, it is less than 3 percent to the communication, including feature extraction, user profile generation, and meta data computation. We note that the overall costs are only a small portion of costs for uploading and downloading images with several MBs. As a result, our security design can be adopted to existing mobile social media applications with little client overhead introduced.

5.2.2 Index Space Overhead

Fig. 5 compares the space overhead between KIK12 and our proposed secure index. We can see that KIK12 suffers from the quadratic growing $O(n^2)$ storage cost, while our index size $O(n)$ is linearly increased with the number of users n. Specifically, KIK12 contains total $l \times n$ buckets after random padding, where l is the number of LSH functions and $l \times n$ is the upper bound of distinct LSH values. Here each LSH bucket stores an n-bit binary vector to indicate which of the total n users have the same LSH value associated with the

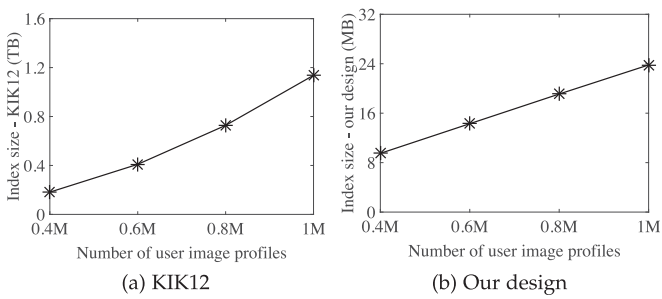


Fig. 5. Space overhead comparison.

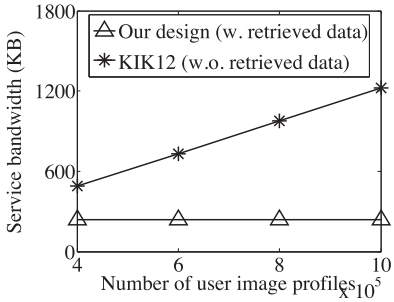


Fig. 6. Bandwidth overhead.

bucket. All n-bit LSH buckets are encrypted via symmetric encryption. Thus, the total size of the index in KIK12 is equal to $l \times n^2/8$ byte. The size of our index design is $u \times n/\tau$ byte, where τ is the index load factor and u is the bit length of one encrypted bucket B_i, i.e., 32 byte as the output of SHA-2. In Fig. 5a, all space sizes for different numbers of user image profiles are reported under parameters with $l = 10$ and $\tau = 80\%$. For $n = 1$ million, the index of KIK12 is around 1.13 TB, while ours consumes only 38 MB.

5.2.3 Index Building Cost

SF can build our secure index in less than 1 minute for 1 million users as shown in Fig. 7. However, when the index load factor increases, more cuckoo kick-away operations will be triggered. Thus, the time of insertion phase becomes larger when building the secure index as shown in Figs. 7a and 7b. Specific to our dataset, when we set the load factor below 90 percent, we can get a reasonable performance in terms of both index building cost and secure operation latency. For encryption phase, because encrypting each bucket takes roughly the same time, the cost is linear in the total number of buckets as shown in Fig. 7c. For the fixed number of user image profiles, as the index load factor increases, the index becomes more and more compact. Thus, the encryption phase takes less time. In Fig. 7d, if the number of kick-way operations is excessively large, the timing cost of insertion phase will dominate the index building cost.

5.2.4 Service Bandwidth Overhead

Fig. 6 compares the service bandwidth cost between KIK12 and our design. In KIK12, one sends the search query with l trapdoors, and retrieves l encrypted binary vectors, each with n bits, resulting in total $l \times n/8$ byte. It grows linearly as the number of users increases, which is not scalable for social media applications with millions of users. In our design, the query incurs constant bandwidth cost with respect to $l \times (d+1)$ secure query trapdoors (32 bytes each) and retrieved encrypted user image profiles (4 KB for the 1,000-dimensional vector). For $l = 10$ and $d = 5$, the bandwidth overhead is around 236 KB for the total 50 trapdoors and retrieved ciphertexts. As shown, even without the retrieved ciphertexts, KIK12 introduces 1,220 KB service bandwidth, 5× more than ours, when n is 1 million.

5.2.5 Query Latency Evaluation

The query operation latency excluding the network delay is measured in Fig. 8. We measure the operation latency under the revised index design which supports secure deletion

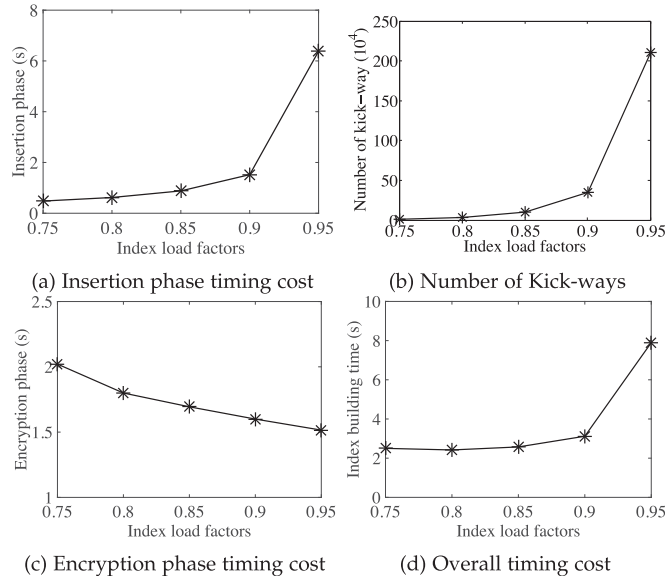


Fig. 7. Secure index building time evaluation.

and secure insertion. The performance is evaluated for 100 operations under the parameter setting $l = 10$ and $d = 5$ over secure index with different load factors. The social discovery query latency and the deletion latency are both constant as they only require one round interaction between *SF* and *CS* with a constant number of accessed buckets determined by l and d. But as the secure deletion needs to further update the secure index via bucket re-masking (see Section 3.4), its latency is slightly larger than the latency of social discovery query. As shown, the latency for 100 queries is around 10ms and the latency for 100 deletions is around 20 ms. They are indeed practical.

The insertion may involve multiple round interactions, so its latency is highly related to the number of kick-away operations. Fig. 8 also reports the total number of kick-way operations for 100 insertions of new data after 1 million data is already in the index. Our experiment shows that the average number of kick-away operations per insertion can

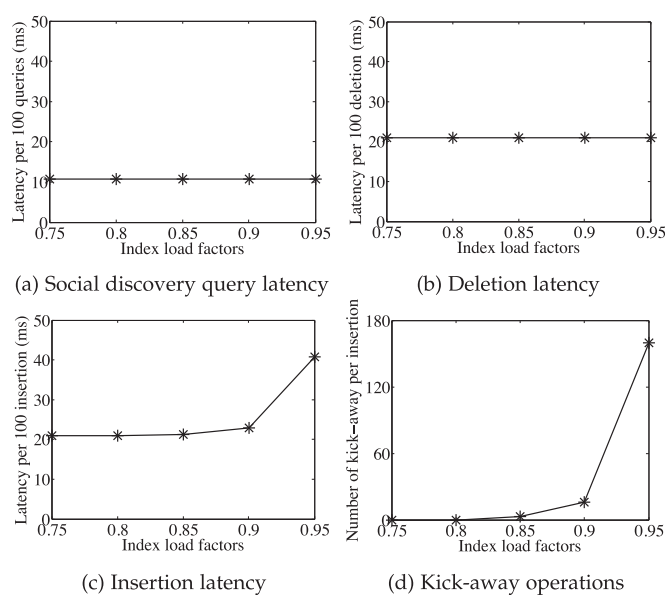
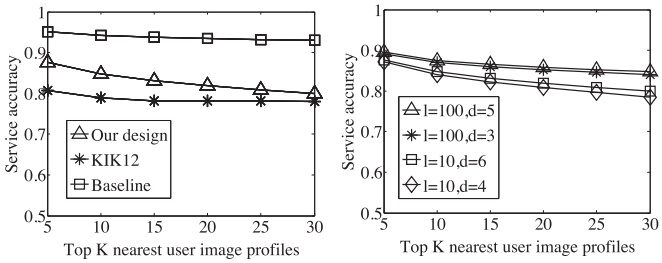


Fig. 8. Operation performance evaluation.



(a) Accuracy comparison (b) Accuracy measurement

Fig. 9. Service accuracy evaluation.

be adjusted by setting the LSH parameters and index load factors. For the index with 1 million users already, only less than 1 kick-away per insertion is triggered on average when $\tau \leq 85\%$, $l = 10$, $d = 5$. Therefore, the latency is roughly the same as the latency of deletion. When the number of kick-away operations increases, the latency of insertion will increase accordingly since each kick-away interaction re-encrypts the accessed buckets.

5.2.6 Service Accuracy Measurement

We compare our discovery accuracy with KIK12 as well as a baseline approach. The baseline approach denotes the basic flow of using LSH for plaintext similarity search as introduced in Section 2.2. For ranking, the distance measurement is performed over the retrieved data from the inverted index. While in KIK12, each item from the candidate set is directly ranked based on its occurrences in common LSH buckets. Our accuracy measurement from the average of 100 query user image profiles $\{S_q\}$ is shown in Fig. 9. Such measurement $\frac{1}{K}\sum_{i=1}^{K}\frac{||S'_i - S_q||}{||S_i - S_q||}$ has been adopted in an early work [29]. It measures the closeness between the Euclidean distance of top-K image profiles $\{S_i\}$ and the ground truth $\{S'_i\}$ via linearly scanning all the profiles.

Fig. 9a shows that the accuracy of our results is higher than the accuracy of the score-based ranking method but lower than the accuracy of baseline for $l = 10$ and $d = 5$. For baseline approach, it requires the distance comparison over a very large set of potential candidates (around 5,000 according to our test). The larger set of candidates explains the better accuracy of the experiment result. For KIK12, the comparison shows that the number of occurrences in common LSH buckets is not always a good indicator for direct similarity ranking. The accuracy of our design is higher, also because *cuckoo-kick* can help to improve the retrieval accuracy. Specifically, *cuckoo-kick* refines the identifiers in relevant buckets so as to make retrieved user profiles close to the queried one. Similar profiles can hardly be moved away from a group of similar profiles while distant profiles are very likely kicked apart. Intuitively, increasing the number of *cuckoo-kick* may filter more distant profiles. Yet, it will increase the cost of index building. We argue that it is acceptable since the cost is one-time.

Fig. 9b further demonstrates the impact of system parameters l and d on the accuracy of our design. Basically, the more user profiles are retrieved, the more candidates can be obtained for ranking, leading to better accuracy measure but at the cost of bandwidth and latency. In this experiment, the results indicate that retrieving 60 profiles has 5 percent loss of accuracy compared to retrieving 300 profiles, but can save 4 times bandwidth and query cost. As we can see, those parameters should carefully be selected based on the priority between accuracy and efficiency to fulfill the service quality requirement in practice.

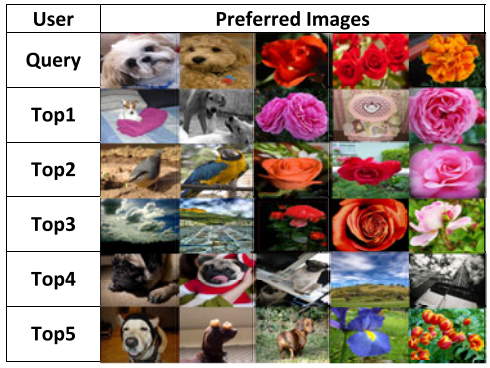


Fig. 10. Social discovery results: the preferred images of one target user and the recommended top five users are exhibited respectively. It shows the effectiveness of our "visual word" based image profile matching, which indicates that those users share mutual interests like flowers and dogs.

5.2.7 Visual Quality Evaluation

To evaluate our service quality of image-centric social discovery, we first select one user, who has images of flowers and dogs to represent her social interests. Then, our service returns top five users with image profiles that are closest to this target user, among our created 1 million users. Explicitly, the result is shown in Fig. 10. Those preferred images of recommended users are visual similar with the images of the target user. They have either flowers or dogs or both. Therefore, the recommended users can be considered as the potential social discovery candidates with mutual social interests to the target user. This indicates the visual consistency of our social discovery with human perception. Note that our security design does not bring inaccuracy because the one-wayness of PRF makes the similar users still to be grouped together as discussed in Section 3.2. The visual quality is determined by the quality of user image profiles and the quality of image features.

6 Related Work

Content-Based Friend Recommendation. Friend recommendation is one of the fundamental social network services. With the emerging popularity of social media, media content such as images have been used for self and social disclosure. One related approach defines the minimum cosine distance of two users' image features as user's visual similarity to assist ranking friend candidates [3]. Another one called content-matching measures the distances of user feature vectors derived from user images [2]. Despite very useful, all these works do not consider the privacy protection on user images. In our research, we provide strong security guarantee for social discovery, which is conducted based on encrypted user images and the related encrypted user image profiles.

Privacy-Preserving Social Profile Attribute Matching. Our research also relates to a branch of work on privacy-preserving profile matching [11], [12] that typically adopts one of

two approaches. One approach is private set intersection (PSI) and private cardinality of set intersection (PCSI) [11]. They treat each user's profile as a set of attributes and compute the attribute set intersection of two users against a certain threshold to decide if they are potential friends. The other approach treats a user's profile as a vector [12]. Two users are considered as potential friends when their secure scalar product is under a certain threshold.

However, all these approaches treat users' profiles as sets of numbers or even coarse-grained binary values, which do not necessarily express the versatile details of users' interests and/or experiences. Thus, it could be difficult to consistently provide good quality social discovery. Also, they incur the computation/communication burden of friend finding directly to individual users, which might not be practically scalable with millions of users. Compared to them, our design focuses on the image-centric approach with built-in security, and offloads the storage and processing burden directly to cloud.

Search over Encrypted Data. Privacy-preserving social discovery over encrypted image content is also closely related to the works on symmetric searchable encryption [17], [18], [19], [31], [33] and secure ranked search [39], [40], which support efficient and secure search over encrypted data. Yet, all these schemes only support textual files for exact keyword search, but are not necessarily suitable for our target context. One recent work studied similarity search over encrypted data [28], which builds on the direct combination of searchable encryption and locality-sensitive hashing based index. As mentioned in Section 3.2, the index design suffers from the unscalable storage cost, and has low space efficiency. Besides, it does not explicitly support data dynamics.

In database community, related works [41], [42] on similar topic is built on the metric index and special partition, both through computing distances with private pre-defined references. However, the security strength is limited, as the private references are subject to leakage when the attacker is able to adaptively exploit the search request generator. One arguably stronger design on an encrypted hierarchical index was proposed in [41], but it is subject to high bandwidth overhead and incurs considerable local burden for searching. Another work [43] finds nearest signals through secure multi-party computation on various distance metrics without knowing the content. The model is different from ours. And these protocols introduce additional computation and bandwidth cost due to the relatively expensive public key encryption adopted. In [44], [45], the proposed designs target on private approximate matches over encrypted databases via random transformation techniques. However, both designs require a linear scan on the whole dataset, and thus are hardly scalable for large-scale datasets.

In signal processing community, researchers have also worked on secure multimedia similarity evaluation [46], which introduces protocols to encrypt the feature vectors in a distance-preserving fashion for direct similarity comparison. However, the designs are missing formal rigorous security treatment [47], and the order-preserving encryption [48] utilized there is known to provide a limited level of security.

Difference from Conference Version. Part of the design in this paper have previously appeared in [1]. We have added more technical contributions and revised the article greatly. The primary improvements are as follows. First, we enhanced the system implementation and evaluation, deploying the client to an android phone and the backend to the real cloud environment (Amazon EC2). The energy and computation costs on the mobile client are measured, and the service evaluation is enhanced by utilizing newly trained parameters with optimized codes. Second, we elaborate on the user image profile generation and the secure updates of user image profiles. Third, we generalize the formal security proofs about the secure updates, and conduct more discussions on the defined leakage.

7 Conclusion

In this paper, we propose and implement a privacy-preserving image-centric social discovery service to suggest potential friends for users with common interests with privacy protection. We model users' preferences in terms of user image profile vectors in visual Bag-of-Words representation, and design a secure and compact similarity index to facilitate privacy-preserving similarity search. The proposed index is scalable and allows similarity search over millions of encrypted user profile vectors. In addition, we address service-driven requirements on user image update and encrypted image sharing. We carry out experiments on an Android phone and the Amazon EC2 platform, demonstrating that our service achieves practical performance under a large image dataset and 1 million user profiles. Our social discovery results are of high-quality and consistent with human perception.

Acknowledgments

This work was supported in part by Research Grants Council of Hong Kong (Projects No. CityU 138513 and No. CityU 11276816), the Natural Science Foundation of China under Project 61572412, Innovation and Technology Commission of Hong Kong under ITF Project ITS/307/15, and the AWS Education Research Grant. Portions of Dr. Kui Ren's research was supported by US National Science Foundation under grant CNS-1262277. Portions of Dr. Squicciarini's work was supported by US National Science Foundation under grant NSF 1421776. The preliminary result of this paper was presented at the 34th IEEE Int'l Conference on Distributed Computing Systems (ICDCS) 2014 [1].

References

[1] X. Yuan, X. Wang, C. Wang, A. C. Squicciarini, and K. Ren, "Enabling privacy-preserving image-centric social discovery," in *Proc. IEEE 34th Int. Conf. Distrib. Comput. Syst.*, 2014, pp. 198–207.

[2] J. Chen, W. Geyer, C. Dugan, M. Muller, and I. Guy, "Make new friends, but keep the old: Recommending people on social networking sites," in *Proc. SIGCHI Conf. Human Factors Comput. Syst.*, 2009, pp. 201–210.

[3] T. Yao, C.-W. Ngo, and T. Mei, "Context-based friend suggestion in online photo-sharing community," in *Proc. 19th ACM Int. Conf. Multimedia*, 2011, pp. 945–948.

[4] R. Negoescu and D. Gatica-Perez, "Analyzing Flickr groups," in *Proc. Int. Conf. Content-Based Image Video Retrieval*, 2008, pp. 417–426.

[5] G. Cubie, "Social discovery sites' influence on retail expanding," 2012. [Online]. Available: http://www.thedrum.com/news/2013/10/02/social-discovery-sites-influence-retail-expanding-rakutens-playcom-numbers-find

[6] Instagram, "Press news," 2016. [Online]. Available: https://www.instagram.com/press/?hl=en
[7] Instagram Engineering Blog, "What powers Instagram: Hundreds of instances, dozens of technologies," 2015. [Online]. Available: http://instagram-engineering.tumblr.com/post/13649370142/what-powers-instagram-hundreds-of-instances-dozens-of
[8] M. Smith, C. Szongott, B. Henne, and G. von Voigt, "Big data privacy issues in public social media," in *Proc. 6th IEEE Int. Conf. Digit. Ecosystems Technol.*, 2012, pp. 1–6.
[9] K. Ren, C. Wang, and Q. Wang, "Security challenges for the public cloud," *IEEE Internet Comput.*, vol. 16, no. 1, pp. 69–73, Jan. 2012.
[10] C. Savage, C. Miller, and N. Perlroth, "N.S.A. said to tap Google and Yahoo abroad," 2013. [Online]. Available: http://www.nytimes.com/2013/10/31/technology/nsa-is-mining-google-and-yahoo-abroad.html
[11] M. Nagy, E. De Cristofaro, A. Dmitrienko, N. Asokan, and A. Sadeghi, "Do i know you?-efficient and privacy-preserving common friend-finder protocols and applications," in *Proc. 29th Annu. Comput. Secur. Appl. Conf.*, 2013, pp. 159–168.
[12] W. Dong, V. Dave, L. Qiu, and Y. Zhang, "Secure friend discovery in mobile social networks," in *Proc. IEEE INFOCOM*, 2011, pp. 1647–1655.
[13] M. McPherson, L. Smith-Lovin, and J. Cook, "Birds of a feather: Homophily in social networks," *Annu. Rev. Sociology*, vol. 27, pp. 415–444, 2001.
[14] J. Yang, Y. Jiang, A. Hauptmann, and C. Ngo, "Evaluating bag-of-visual-words representations in scene classification," in *Proc. ACM Workshop Multimedia Inf. Retrieval*, 2007, pp. 197–206.
[15] A. Andoni and P. Indyk, "Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions," *Commun. ACM*, vol. 51, pp. 117–122, 2008.
[16] R. Pagh and F. Rodler, "Cuckoo hashing," *J. Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
[17] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 79–88.
[18] P. Van Liesdonk, S. Sedghi, J. Doumen, P. Hartel, and W. Jonker, "Computationally efficient searchable symmetric encryption," in *Proc. 7th VLDB Workshop Secure Data Manage.*, 2010, pp. 87–100.
[19] S. Kamara, C. Papamanthou, and T. Roeder, "Dynamic searchable symmetric encryption," in *Proc. 1st ACM Conf. Comput. Commun. Secur.*, 2012, pp. 965–976.
[20] M. Huiskes and M. Lew, "The MIR flickr retrieval evaluation," in *Proc. ACM Int. Conf. Multimedia Inf. Retrieval*, 2008, pp. 39–43.
[21] R. Baden, A. Bender, N. Spring, B. Bhattacharjee, and D. Starin, "Persona: An online social network with user-defined privacy," in *Proc. ACM SIGCOMM Conf. Data Commun.*, 2009, pp. 135–146.
[22] S. Yu, C. Wang, K. Ren, and W. Lou, "Achieving secure, scalable, and fine-grained access control in cloud computing," in *Proc. 29th Conf. Inf. Commun.*, 2010, pp. 534–542.
[23] R. Panigrahy, "Entropy based nearest neighbor search in high dimensions," in *Proc. 17th Annu. ACM-SIAM Symp. Discrete Algorithm*, 2006, pp. 1186–1195.
[24] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, "Multi-probe LSH: Efficient indexing for high-dimensional similarity search," in *Proc. 33rd Int. Conf. Very Large Data Bases*, 2007, pp. 950–961.
[25] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, pp. 91–110, Nov. 2004.
[26] H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool, "Speeded-up robust features (SURF)," *Comput. Vis. Image Understanding*, vol. 110, no. 3, pp. 346–359, 2008.
[27] G. Qian, S. Sural, Y. Gu, and S. Pramanik, "Similarity between Euclidean and cosine angle distance for nearest neighbor queries," in *Proc. ACM Symp. Appl. Comput.*, 2004, pp. 1232–1237.
[28] M. Kuzu, M. Islam, and M. Kantarcioglu, "Efficient similarity search over encrypted data," in *Proc. IEEE 28th Int. Conf. Data Eng.*, 2012, pp. 1156–1167.
[29] Y. Hua, B. Xiao, and X. Liu, "NEST: Locality-aware approximate query service for cloud computing," in *Proc. IEEE INFOCOM*, 2013, pp. 1303–1311.
[30] B. Logan, "People you mayknow," 2008. [Online]. Available: https://www.facebook.com/notes/facebook/people-you-may-know/15610312130/
[31] D. Cash, et al., "Dynamic searchable encryption in very large databases: Data structures and implementation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014.
[32] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable symmetric encryption with small leakage," in *Proc. Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, pp. 23–26.
[33] S. Kamara and C. Papamanthou, "Parallel and dynamic searchable symmetric encryption," in *Proc. Int. Conf. Financial Cryptography Data Secur.*, 2013, pp. 258–274.
[34] M. Islam, M. Kuzu, and M. Kantarcioglu, "Access pattern disclosure on searchable encryption: Ramification, attack and mitigation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2012.
[35] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 668–679.
[36] OpenCV, "OpenCV for android SDK," 2016. [Online]. Available: http://opencv.org/platforms/android.html
[37] A. Andoni, "E2LSH package," 2016. [Online]. Available: http://web.mit.edu/andoni/www/LSH
[38] J.-H. Borth, "Power Tutor 2 Pro," 2016. [Online]. Available: https://play.google.com/store/apps/details?id=edu.umich.PowerTutor2&hl=en
[39] C. Wang, N. Cao, K. Ren, and W. Lou, "Enabling secure and efficient ranked keyword search over outsourced cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 8, pp. 1467–1479, Aug. 2012.
[40] N. Cao, C. Wang, M. Li, K. Ren, and W. Lou, "Privacy-preserving multi-keyword ranked search over encrypted cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 1, pp. 222–233, Jan. 2014.
[41] M. Yiu, I. Assent, C. Jensen, and P. Kalnis, "Outsourced similarity search on metric data assets," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 2, pp. 338–352, Feb. 2012.
[42] B. Yao, F. Li, and X. Xiao, "Secure nearest neighbor revisited," in *Proc. IEEE 29th Int. Conf. Data Eng.*, 2013, pp. 733–744.
[43] C. Blundo, E. De Cristofaro, and P. Gasti, "EsPRESSo: Efficient privacy-preserving evaluation of sample set similarity," in *Proc. 7th Int. Workshop Data Privacy Manage.*, 2013, pp. 89–103.
[44] W. Du and M. Atallah, "Protocols for secure remote database access with approximate matching," in *Proc. 1st Workshop Secur. Privacy E-Commere*, 2001, pp. 87–111.
[45] W. Wong, D. Cheung, B. Kao, and N. Mamoulis, "Secure kNN computation on encrypted databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2009, pp. 139–152.
[46] W. Lu, A. Swaminathan, A. L. Varna, and M. Wu, "Enabling search over encrypted multimedia databases," *Proc. SPIE Media Forensics Secur.*, vol. 7254, 2009, Art. no. 725418.
[47] Z. Qin, J. Yan, K. Ren, C. W. Chen, and C. Wang, "Towards efficient privacy-preserving image feature extraction in cloud computing," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 497–506.
[48] A. Boldyreva, N. Chenette, and A. ONeill, "Order-preserving encryption revisited: Improved security analysis and alternative solutions," in *Proc. 31st Annu. Conf. Advances Cryptology*, 2011, pp. 578–595.

Xingliang Yuan received the BS degree in electrical engineering from Nanjing University of Posts and Telecommunications, in 2008, and the MS degree in electrical engineering from Illinois Institute of Technology, in 2009. He is currently working toward the PhD degree in the Department of Computer Science, City University of Hong Kong. His research interests include cloud computing security, mobile security, and multimedia security. He is a student member of the IEEE.

Xinyu Wang received the BE degree from East China Jiaotong University, in 2011. He is currently working toward the PhD degree in the Department of Computer Science, City University of Hong Kong. He worked as a software engineer with Tencent, Shenzhen, China, from 2011 to 2013. His research interests include cloud computing, network security, and big data.

Cong Wang received the BE and ME degrees in electrical and computer engineering from Wuhan University, in 2004 and 2007, and the PhD degree in electrical and computer engineering from Illinois Institute of Technology, in 2012. He is an assistant professor in the Computer Science Department, City University of Hong Kong. He worked with Palo Alto Research Center, in the summer of 2011. His research interests include cloud and network security, computer and mobile social networks, as well as secure and practical systems. He is a member of the IEEE.

Anna Cinzia Squicciarini received the PhD degree in computer science from the University of Milan, Italy, in 2006. She is an associate professor in the College of Information Science and Technology, Pennsylvania State University. During the years of 2006-2007, she was a post doctoral research associate with Purdue University. Her main interests include access control for distributed systems, privacy, security for Web 2.0 technologies, and grid computing. She is the author or coauthor of more than 60 articles published in refereed journals, and in proceedings of international conferences and symposia. She is a member of the IEEE.

Kui Ren received the PhD degree from Worcester Polytechnic Institute, Worcester, Massachusetts. He is currently a professor of computer science and engineering and the director of the UbiSeC Laboratory, University at Buffalo, Buffalo, New York. His research has been supported by the US National Science Foundation, DoE, AFRL, MSR, and Amazon. He has authored or coauthored more than 160 peer-reviewed journal and conference papers. His current research interests include cloud and outsourcing security, wireless and wearable system security, and human-centered computing. He received the NSF CAREER Award in 2011 and the Sigma Xi/IIT Research Excellence Award in 2012. He also received several Best Paper Awards, including the IEEE ICNP 2011. He serves/has served as associate editors of the *IEEE Transactions on Dependable and Secure Computing*, the *IEEE Transactions on Mobile Computing*, the *IEEE Transactions on Information Forensics and Security*, the *IEEE Wireless Communications Magazine*, the *IEEE Internet of Things Journal*, the *IEEE Transactions on Smart Grid*, the Elsevier's *Pervasive and Mobile Computing*, and Oxford's *The Computer Journal*. He is a member of the Association for Computing Machinery, a distinguished lecturer of the IEEE Vehicular Technology Society, and a fellow of the IEEE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.